



East West University

Course Code & Name: CSE 246 (Algorithms)

Course Instructor: Sanzana Karim Lora

Course Section: 10

Academic Session: Summer 2025

Project Report

Group No.- 04

Group Members:

Name	Id
MD Abir Rahman	2023-3-60-223
Inteshar Alam	2023-3-60-466
Abdullah Hussain Rida	2023-3-60-225
Tahameed Karim	2023-3-60-373

Project Title:

****ByteTrip: Code your route, Byte your trip.****

Submission Date & Day: September 04, 2025 (Thursday)

Table of Content

No.	Content Name	Page No.
1.	Problem Statement	4
2.	System Requirements	5
3.	System Design	6-7
4.	Implementation	8
5.	Testing Result	9-11
6.	Future Scope	12

Optimizing Public Bus Travel Across All Districts in Bangladesh



Problem Statement

Navigating inter-district public transport in Bangladesh can be confusing for travelers and daily commuters alike. Whether it's a student traveling from Khulna to Dhaka, or a trader heading from Rajshahi to Sylhet — most passengers are unaware of the cheapest or fastest routes, especially when multiple transfers are involved or when indirect routes offer better savings.

The absence of a digital, optimized route planner leads to inefficient travel decisions — causing higher travel costs, longer journey times, and overall inconvenience. Commuters often rely on guesswork or word of mouth, which results in unnecessary route switches, overpriced tickets, and hours wasted on suboptimal paths. While GPS apps offer road directions, they lack the intelligence to factor in public transport fare policies, route constraints— making them unreliable for real-world inter-district bus travel.

The goal of this project is to design and implement an intelligent **Inter-District Route Planner** for Bangladesh's public transport system. This system aims to help tourists, students, traders, and travelers make better route decisions by providing optimized travel suggestions based on **cost, time, and distance (hops)**.

We have implement an intelligent web-based tool to find the shortest, cheapest, and time reducing paths across Bangladesh.

System Requirements

Processor:

Minimum: Intel Core i3 or equivalent

Recommended: Intel Core i5/i7 or AMD Ryzen series

RAM:

Minimum: 4 GB

Recommended: 8 GB or higher

Operating System:

Windows

Linux

macOS

IDE / Tools:

Visual Studio Code (preferred)

Python 3.8 or higher

Flask framework installed

Pandas library installed

Web browser (Google Chrome / Mozilla Firefox / Edge)

System Design

The system is designed to determine the **optimal routes between districts** using two graph-based algorithms: **Dijkstra's Algorithm** and **Breadth-First Search (BFS)**. The dataset is stored in a **CSV file** containing district-to-district connections with attributes:

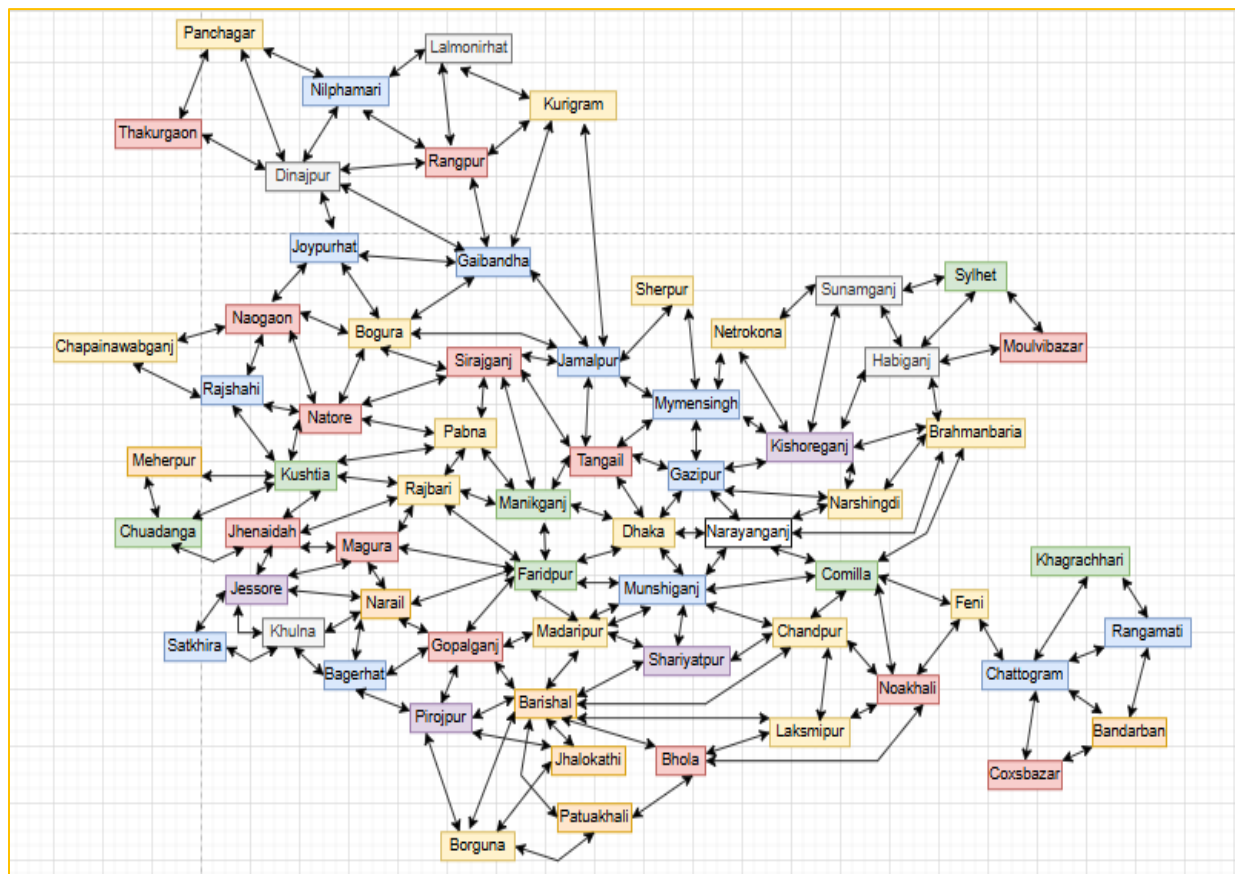
-Travel Time (minutes)

-Travel Distance (km)

-Travel Cost (currency)

These attributes are used as edge weights in the graph.

Visual Representation of our Graph using Diagram



Graph Construction

Nodes: Represent districts.

Edges: Represent direct land connections between districts.

Edge Weights: Extracted from the CSV file (time, distance, cost).

Algorithms

1. Dijkstra's Algorithm

~Handles **weighted graphs** with non-negative edge values.

~Finds the shortest path considering weights.

~Applied **three times**:

-**Minimum Time**

-**Minimum Distance**

-**Minimum Cost**

2. Breadth-First Search (BFS)

~Generally used for **unweighted graphs**, but here it is adapted to handle weighted attributes from the CSV file.

~Computes shortest paths in terms of:

- **Time**
- **Distance**
- **Cost**

~Runs parallel to Dijkstra's for comparison.

Implementation

1. Input Stage

- A. Read CSV file using Pandas.
- B. Construct graph with nodes (districts) and edges (connections).

2. Processing Stage

A. Run **Dijkstra's Algorithm for:**

- Shortest Time
- Shortest Distance
- Cheapest Cost

B. Run **BFS for:**

- Shortest Time
- Shortest Distance
- Cheapest Cost

3. Output Stage

A. Collect 6 different route results:

- Dijkstra: Time, Distance, Cost
- BFS: Time, Distance, Cost

B. Compare results between the two algorithms.

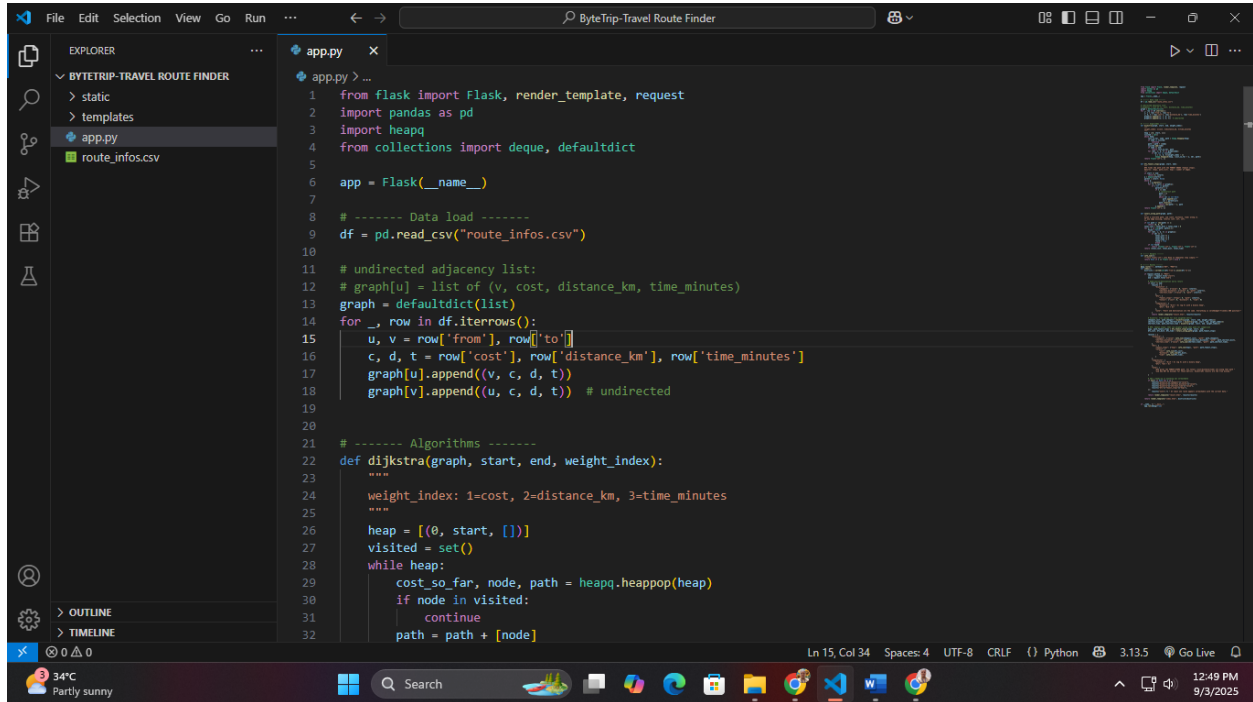
C. Display the **best route recommendations to the user.**

Why Both Algorithms?

1. **Dijkstra's Algorithm** is the standard and efficient choice for weighted graphs.
2. **BFS** allows an alternate approach and validation, ensuring results are not biased toward a single method.
3. Running both provides a **comparative analysis** of performance and accuracy.

Testing Results

This is our app.py file



The screenshot shows a Visual Studio Code editor window titled "ByteTrip-Travel Route Finder". The Explorer sidebar on the left shows a project structure with a folder "BYTETRIP-TRAVEL ROUTE FINDER" containing subfolders "static" and "templates", and files "app.py" and "route_infos.csv". The main editor area displays the content of "app.py".

```
1 from flask import Flask, render_template, request
2 import pandas as pd
3 import heapq
4 from collections import deque, defaultdict
5
6 app = Flask(__name__)
7
8 # ----- Data load -----
9 df = pd.read_csv("route_infos.csv")
10
11 # undirected adjacency list:
12 # graph[u] = list of (v, cost, distance_km, time_minutes)
13 graph = defaultdict(list)
14 for _, row in df.iterrows():
15     u, v = row['from'], row['to']
16     c, d, t = row['cost'], row['distance_km'], row['time_minutes']
17     graph[u].append((v, c, d, t))
18     graph[v].append((u, c, d, t)) # undirected
19
20
21 # ----- Algorithms -----
22 def dijkstra(graph, start, end, weight_index):
23     """
24     weight_index: 1=cost, 2=distance_km, 3=time_minutes
25     """
26     heap = [(0, start, [])]
27     visited = set()
28     while heap:
29         cost_so_far, node, path = heapq.heappop(heap)
30         if node in visited:
31             continue
32         path = path + [node]
```

Open the Terminal and write: `python app.py`

If you haven't installed flask framework & pandas library, Then run this commands:

-- `pip install flask`

-- `pip install pandas`

```
1 from flask import Flask, render_template, request
2 import pandas as pd
3 import heapq
4 from collections import deque, defaultdict
5
6 app = Flask(__name__)
7
8 # ----- Data load -----
9 df = pd.read_csv("route_infos.csv")
10
11 # undirected adjacency list:
12 # graph[u] = list of (v, cost, distance_km, time_minutes)
13 graph = defaultdict(list)
14 for _, row in df.iterrows():
15     u, v = row['from'], row['to']
16     c, d, t = row['cost'], row['distance_km'], row['time_minutes']
17     graph[u].append((v, c, d, t))
18     graph[v].append((u, c, d, t))
19
20 if __name__ == '__main__':
21     app.run(debug=True)
```

PS D:\CSE246 Final\ByteTrip-Travel Route Finder> python app.py

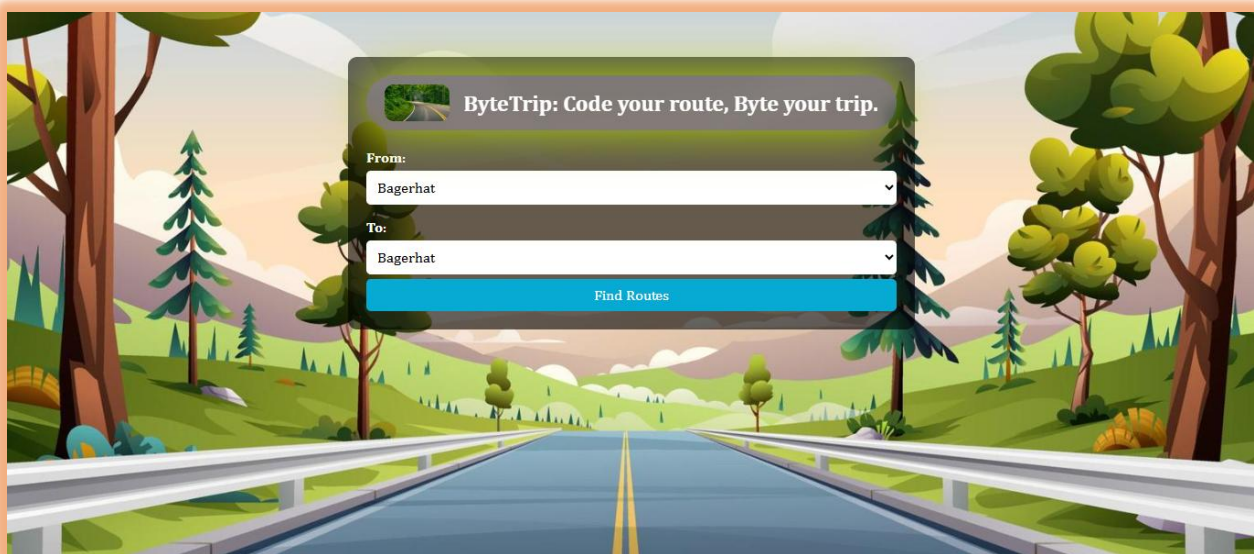
- * Serving Flask app 'app'
- * Debug mode: on
- WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
- * Running on http://127.0.0.1:5000
- Press CTRL+C to quit
- * Restarting with stat
- * Debugger is active!
- * Debugger PID: 114-886-343

Here, we can see that, our website is running at:

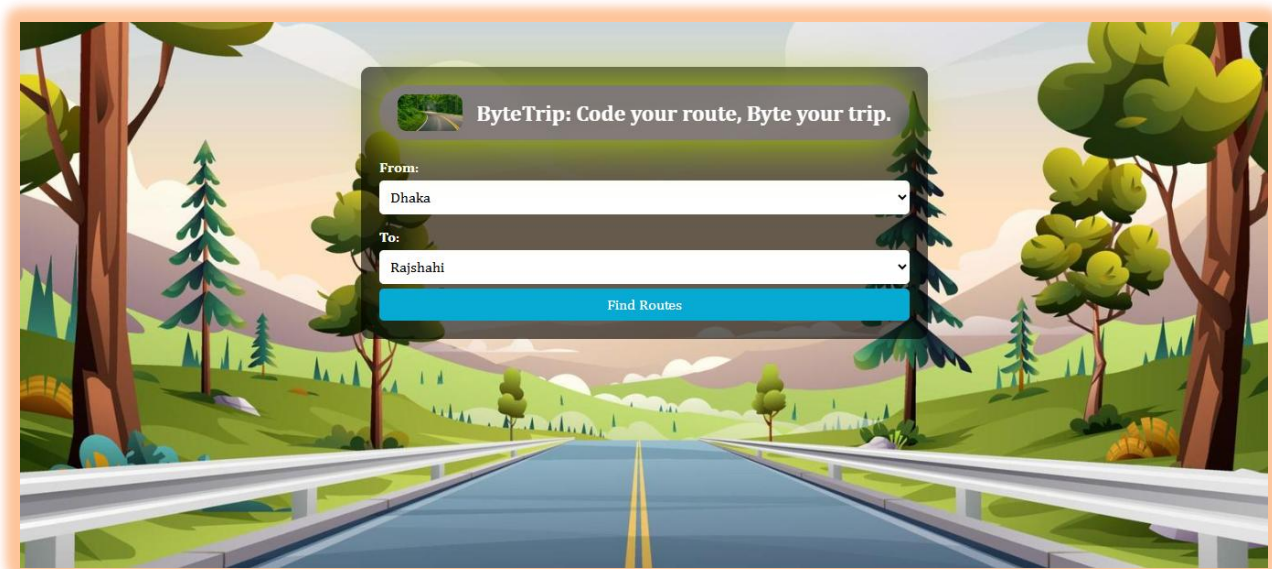
<http://127.0.0.1:5000>

By clicking on that link, we can visit our website.

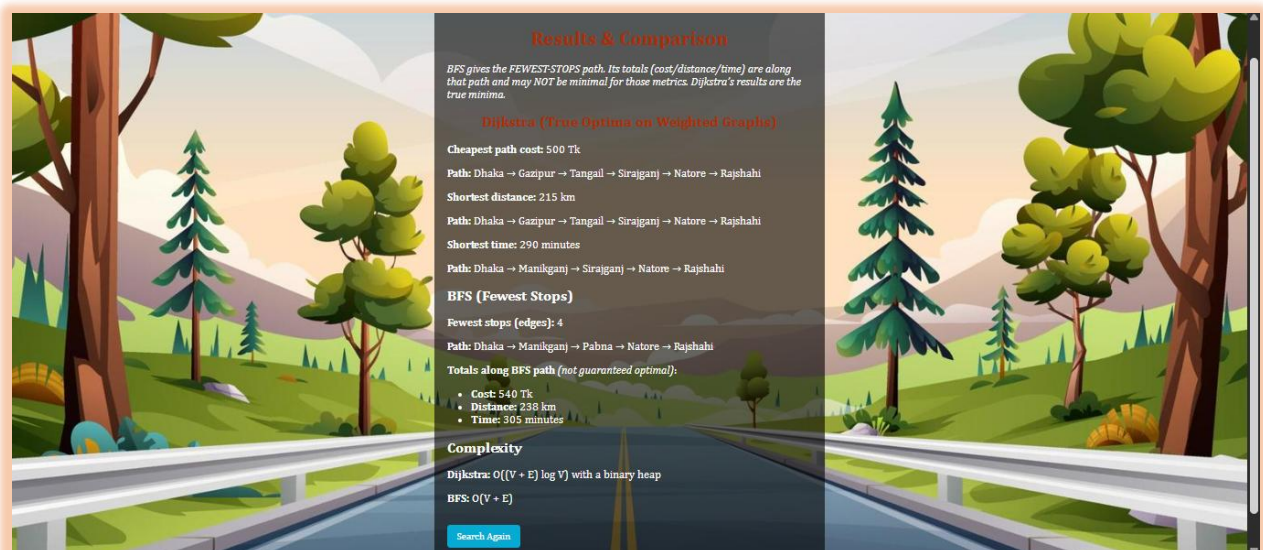
This is our website.



Suppose, we want to know the shortest path between Dhaka to Rajshahi



Here we can see the Cheapest path, Shortest distance and Shortest time between Dhaka to Rajshahi using both Dijkstra's algorithm & BFS. We have also shown the Time Complexity comparison.



Future Scope

While the current system effectively finds optimal inter-district routes, it relies on a static dataset and does not reflect real-time changes like traffic, delays, or fare updates.

Integrating live data from transport operators or government APIs can make the route suggestions more accurate and responsive to real-world conditions.

The current system supports only land-based public transport (buses). Expanding it to include other modes like trains, ferries, and domestic flights would make it more comprehensive and useful for a wider range of travelers across Bangladesh.

Adding interactive map-based route visualization and multi-language support (especially Bengali) will improve accessibility and usability for a broader range of users, including locals unfamiliar with English or digital navigation tools.